CUDA Lab 01 — Completed Task Sheet 230109009

Laboratory Experimental Protocol: Applied Massive Parallelism — CUDA Python, Thread Geometries, and Hardware Latency Pitfalls.

Field	Value
Student Name	Seiitkhan Zhannur
Student ID	230109009
Session / Date	08.10.2026
Assigned GPU ID	GPU 0 (Tesla T4)

Section 1: Hardware Telemetry

Values printed by `nvidia-smi` and the Numba inspection script on my instance (Thu Oct 8 07:08:36 2026).

Field	Recorded value
NVIDIA Driver Version	580.82.07
CUDA Version Supported by Driver	13.0
Allocated GPU Name	Tesla T4
Total Available VRAM	15360 MiB
Compute Capability Major.Minor	7.5
Max Threads Per Block	1024

Section 2: The Crossover Experiment

The GPU starts winning at N = 10,000; below that the CPU is faster.

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner
100	0.071	0.722	0.093	CPU
1,000	0.614	1.636	0.167	CPU
10,000	6.107	0.690	0.092	GPU
100,000	60.665	1.029	0.111	GPU

Array Size (N)	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner
1,000,000	571.673	6.709	0.180	GPU
5,000,000	nan (skipped)	15.635	0.344	GPU
10,000,000	nan (skipped)	33.409	0.541	GPU

For 5,000,000 and 10,000,000 the benchmark code skips the CPU loop. Scaling the 1,000,000 row gives about 2,860 ms and 5,720 ms, so the GPU wins both.

Q1. Empirical crossover point N*: N* = 10,000. At N = 1,000 the CPU still wins (0.614 ms vs 1.636 ms). At N = 10,000 the GPU total is faster for the first time (0.690 ms vs 6.107 ms), so the crossover lies between 1,000 and 10,000.

Q2. Why GPU Total is slower than CPU at N = 100: The GPU total was 0.722 ms against 0.071 ms on the CPU, about 10 times slower. The kernel itself took only 0.093 ms. The other 0.629 ms, about 87% of the total, went to overhead: copying the array to the GPU over PCIe, allocating the output buffer in VRAM, and copying the result back. For 100 elements the computation is tiny, so this fixed transfer cost is much larger than the work, and the CPU finishes before the data has even reached the GPU.

Section 3: Thread Geometry and Boundary Memory Corruption

Task 3.1. Observed behavior of the defective kernel: No error and no crash. The cell printed exactly this:

```
Launching defective kernel with 1024 threads on 1000-element array...
Execution finished (or did it crash?). Checking host copy...
Elements processed: 1000. Last element: 999.0
```

No Python `IndexError` and no `CUDA_ERROR_ILLEGAL_ADDRESS` appeared, even after `cuda.synchronize()`. The 24 extra threads (indices 1000 to 1023) wrote 999.0 past the end of the 1000-element array, into VRAM that does not belong to it. The corruption was silent, which makes it more dangerous than a crash.

Task 3.2. Calculation for N = 37 and block size = 8:

Quantity	Value	How
Total Blocks Allocated	5	ceil(37 / 8)
Total Hardware Threads Launched	40	5 × 8
Number of Active Working Threads	37	one per element, global IDs 0 to 36
Number of Idle Guarded Threads	3	40 − 37, global IDs 37, 38, 39

The inspector output confirms it: in block 4, threads 0 to 4 (global IDs 32 to 36) are ACTIVE and threads 5 to 7 (global IDs 37 to 39) are IDLE (GUARDED).

Section 4: The Race Condition (Parallel Reduction)

The naive kernel returned 2.0 instead of 50,000.0 in every trial, losing 49,998 of the 50,000 updates.

Trial #	Target Sum	Naive Kernel Output	Deviation / Lost Updates
Trial 1	50,000.0	2.0	49,998.0
Trial 2	50,000.0	2.0	49,998.0
Trial 3	50,000.0	2.0	49,998.0
Trial 4	50,000.0	2.0	49,998.0
Trial 5	50,000.0	2.0	49,998.0

On my run the output was the same in all five trials. Almost every thread read the old value of `d_total[0]` before any other thread had written its result, so the threads overwrote each other and nearly all additions were lost.

Atomic Sum Output: 50000.0. Does it equal exactly 50,000.0? Yes.

Performance penalty: Naive Kernel Time: 0.190 ms vs. Atomic Kernel Time: 0.336 ms. The atomic kernel was about 1.8 times slower.

Why atomic operations incur a penalty: `cuda.atomic.add` makes the hardware apply updates to the same memory address one at a time. All 50,000 threads target the single element `d_total[0]`, so they have to wait for each other instead of working in parallel, and each failed attempt is retried. The naive kernel is faster only because it skips this coordination, and that is exactly why its answer is wrong.

Section 5: 2D Grid Processing (Radial Vignette)

A 1024 × 1024 canvas with 16 × 16 thread blocks needs 64 × 64 = 4,096 blocks.

Quantity	Value	How
Blocks in X	64	1024 / 16
Blocks in Y	64	1024 / 16
Total Blocks	4,096	64 × 64
Pixel Value at Center Point (512, 512)	1.000	distance from center is 0, so intensity = 1 − 0
Pixel Value at Outer Corner Point (0, 0)	0.000	distance equals the maximum distance, so intensity = 1 − 1

Section 6: Final Verification Checksum

Inputs used: Student ID 230109009, GPU Tesla T4, crossover N* = 10,000.

TOKEN: C4AD983365469756